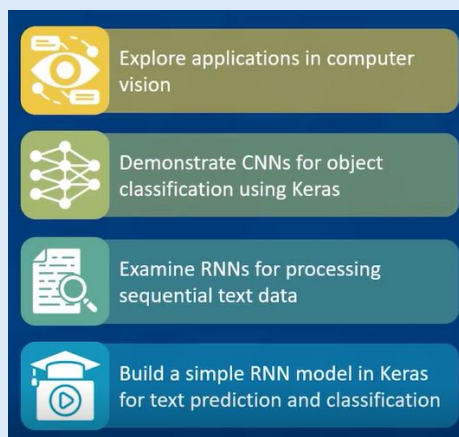


Module 15: Introduction to Deep Learning

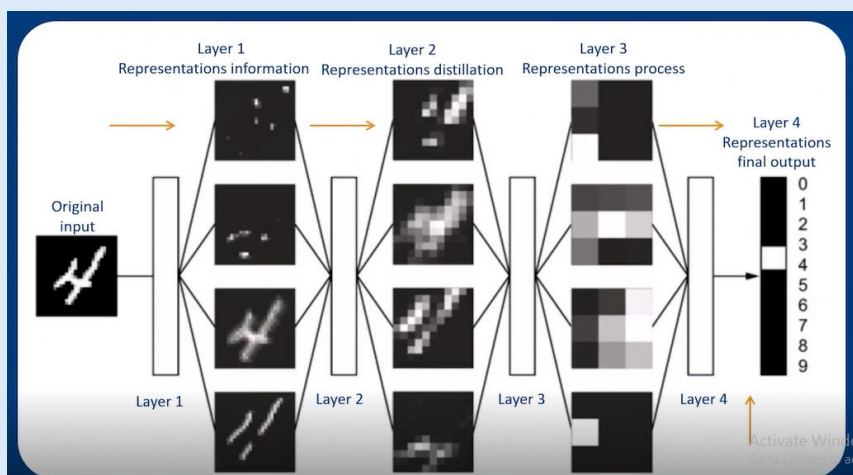
Module Overview

- Deep learning is a key subset of machine learning designed to interpret and process complex data structures efficiently.
- Applications of deep learning span domains such as computer vision, natural language processing, and automated decision-making.
- Convolutional neural networks (CNNs) are used in image classification tasks, identifying patterns and features in visual data.
- Recurrent neural networks (RNNs) are suitable for sequential data like text, supporting tasks such as text generation and classification.
- Practical Keras tutorials demonstrate how CNNs and RNNs function, reinforcing the real-world capabilities of deep learning systems.



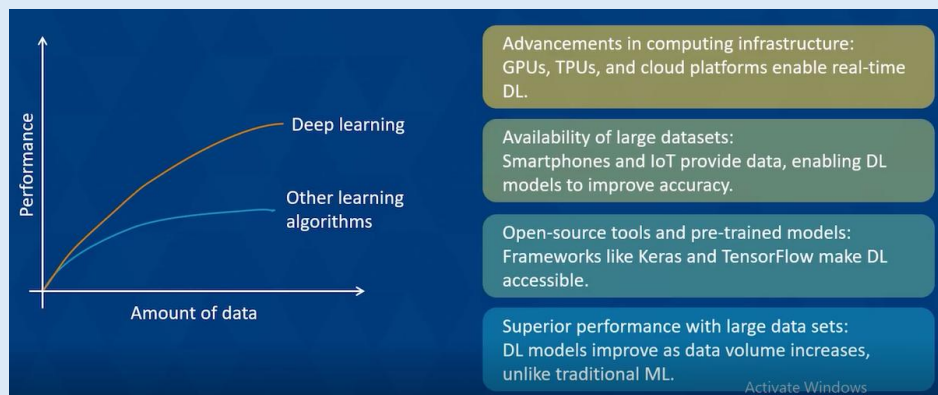
What is Deep Learning?

- Deep learning is a subset of machine learning that processes data through multiple layers to automatically learn complex patterns.



- Unlike traditional models, deep learning eliminates manual feature engineering by learning useful representations directly from raw data.
- Each layer in a deep neural network acts as a filter, refining input data into more abstract and meaningful features.

- Rapid growth in computing power, availability of large datasets, and open-source frameworks have accelerated deep learning's adoption.
- Deep learning models consistently improve with increased data volume, outperforming traditional algorithms in tasks like image and speech recognition.

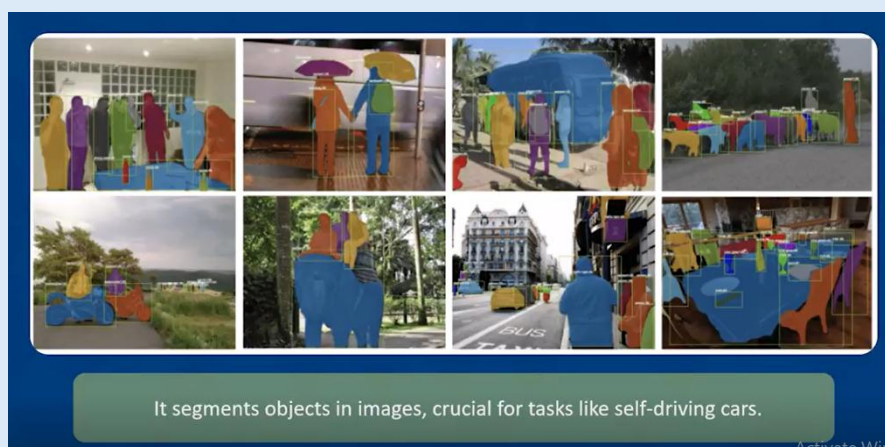


Applications of Deep Learning for Computer Vision

- Object detection enables systems to accurately identify and locate multiple objects within images using bounding boxes.



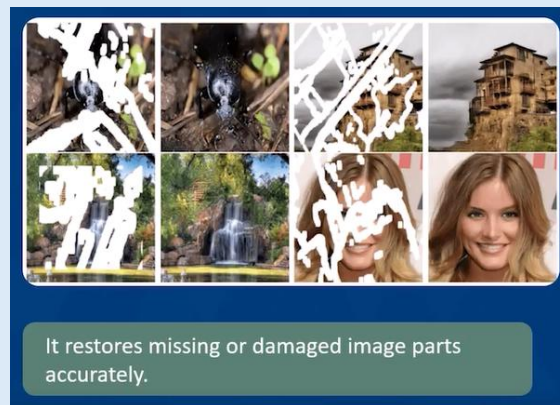
- Image segmentation allows for precise distinction of individual objects within a scene, essential for applications like autonomous driving.



- Style transfer and image colouring transform visual aesthetics by applying artistic styles or converting black-and-white images into colour.



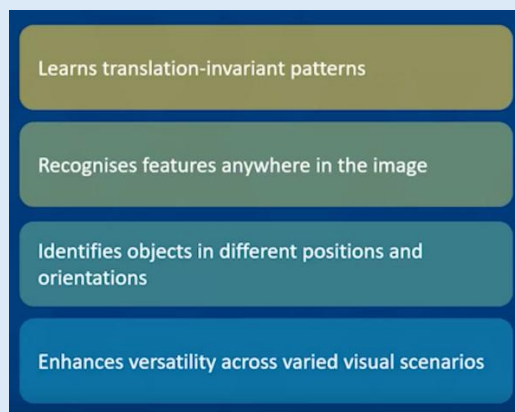
- Image inpainting and super-resolution techniques restore damaged images and enhance low-resolution visuals with realistic details.



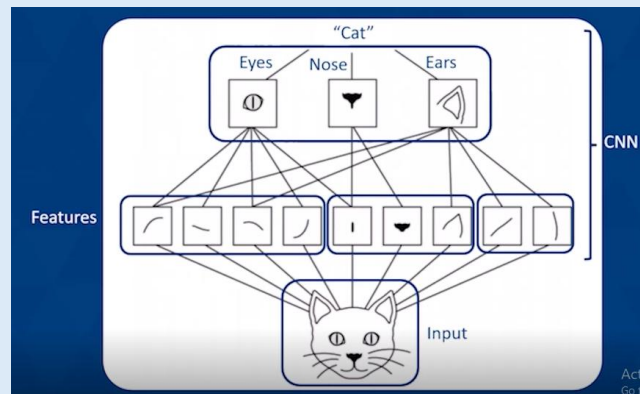
- Generative models and image synthesis create entirely new visual content, demonstrating deep learning's ability to replicate and innovate complex imagery.

Deep Learning for Computer Vision

- Convolutional neural networks (CNNs) are essential for deep learning applications in image search, autonomous vehicles, and classification systems.
- CNNs effectively capture local features in images, enabling them to detect patterns regardless of position or orientation.



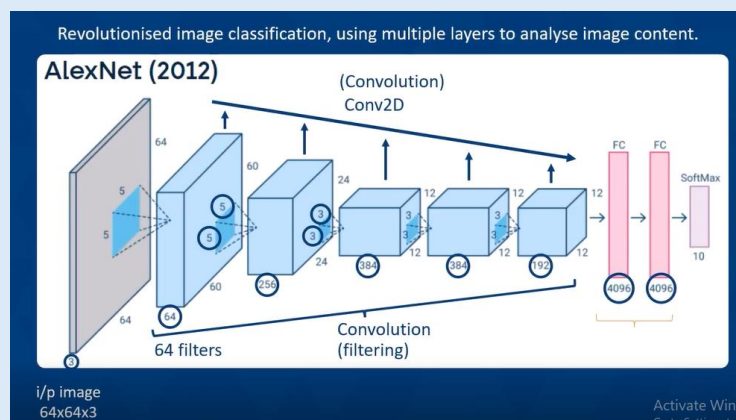
- Spatial hierarchies in CNNs allow for gradual learning of complex objects by combining simple features into higher-level representations.



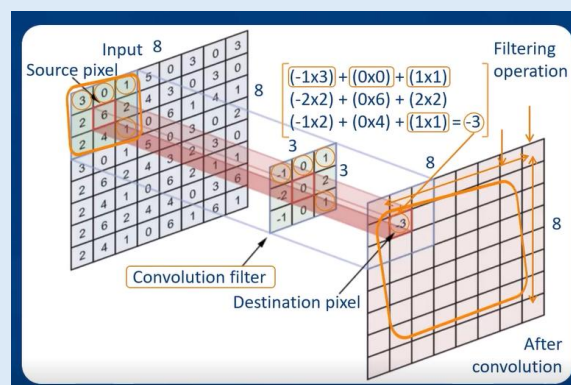
- Unlike densely connected networks, CNNs preserve spatial relationships in data, leading to more efficient learning in visual tasks.
- The layered structure of CNNs builds understanding from basic lines to full objects, mirroring human visual recognition processes.

What is Convolutional Neural Network (CNN)?

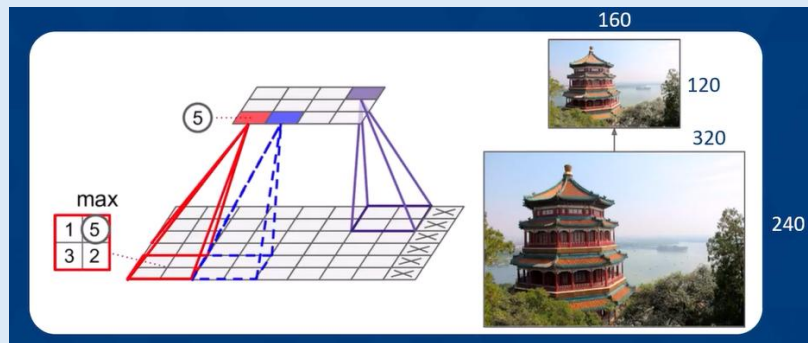
- Convolutional neural networks extract visual features using layers of filters and pooling operations, enabling effective image recognition and classification.



- AlexNet introduced multiple convolutional and dense layers, significantly improving performance on image classification tasks using layered feature extraction.
- Convolutional layers apply filters across images to create feature maps, using operations like padding and strides to manage spatial dimensions.



- Pooling layers reduce the spatial size of feature maps using operations like max or average pooling, lowering computation while preserving key features.



- VGGNet, especially VGG16, enhanced CNN depth with repeated convolution-pooling blocks and fully connected layers, improving accuracy for large-scale image datasets.

Keras Example for Object Classification

- CIFAR-10 dataset provides 60,000 colour images across 10 categories, serving as a practical benchmark for image classification models.
- CNN architecture includes convolutional layers for feature extraction, pooling layers for dimensionality reduction, and dense layers for classification.

```

# 3. Model Creation\n",
"\n",
"Create a CNN model using the Sequential API. The model will consist of\n",
"convolutional layers, max-pooling layers, and dense layers."
},
{
"metadata": {
"id": "q8hKJ4SBTHa"
},
"cell_type": "code",
"source": [
"# Define the CNN model\n",
"model = models.Sequential()\n",
"\n",
"# Add convolutional layers, followed by max-pooling layers\n",
"model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3))) # First conv layer with 32 filters and a 3x3 kernel\n",
"model.add(layers.MaxPooling2D((2, 2))) # Max-pooling layer with a 2x2 pool\n",
"model.add(layers.Conv2D(64, (3, 3), activation='relu')) # Second conv layer\n",
"model.add(layers.MaxPooling2D((2, 2))) # Max-pooling layer\n",
"model.add(layers.Conv2D(64, (3, 3), activation='relu')) # Third conv\n",
"layer\n",
"\n",
"# Add dense layers for classification\n",
"model.add(layers.Flatten()) # Flatten the output of the conv layers\n",
"model.add(layers.Dense(64, activation='relu')) # Fully connected layer with\n",
"64 units\n",
"model.add(layers.Dense(10, activation='softmax')) # Output layer with 10\n",
"units (one for each class) and softmax activation\n",
"\n",
"# Display the model architecture\n",
"model.summary()\n",
],
"metadata": {
"colab": {
"base_url": "https://localhost:8080/"
},
{id": "YmEbFslBSFL",
"outputId": "0d2f930-6ef-4c4d-f734-acb0100c19f6"}
},
"execution_count": 4,
"outputs": [
{
"output_type": "stream",
"name": "stdout",
"text": [
"Model: 'sequential'\n",

```

Our CNN consists of three key layers:

- Convolutional layers: Extract features using filters to detect patterns like edges and textures
- Pooling layers: Reduce spatial dimensions, improving efficiency and feature extraction
- Dense layers: Fully connected layers interpret extracted features for classification

- Model compilation uses Adam optimiser and sparse categorical crossentropy loss, suitable for multi-class classification with exclusive class labels.
- Training is monitored using accuracy and loss plots, showing consistent improvement across epochs for both training and validation data.

```

# 4. Compile and Train the Model\n",
"\n",
"Compile the model with an appropriate optimizer, loss function,\n",
"and metrics. Then, train the model using the training data."
},
{
"metadata": {
"id": "B80p9-uTxm"
},
"cell_type": "code",
"source": [
"# Compile the model\n",
"model.compile(optimizer='adam', # Optimizer for gradient\n",
"               loss='sparse_categorical_crossentropy', # Loss\n",
"               metrics=['accuracy']) # Metric to monitor during\n",
"training\n",
"\n",
"# Train the model\n",
"print('Training model...')\n",
"history = model.fit(x_train, y_train, # Training data and\n",
"                    epochs=epochs, # Number of iterations over\n",
"                    the entire dataset\n",
"                    batch_size=batch_size, # Number of samples\n",
"                    per gradient update\n",
"                    validation_data=(x_test, y_test)) # Data on\n",
"which to evaluate the loss and any model metrics at the end of each\n",
"epoch\n",
],
"metadata": {
"colab": {
"base_url": "https://localhost:8080/"
},
{id": "9-105FqQT01-",
"outputId": "124b3103-65df-46c0-c090-1d90d6ebb81"}
},
"execution_count": 6,
"outputs": [

```

- Uses Adam optimiser for efficient weight updates
- Uses sparse_categorical_crossentropy for multi-class classification
- `model.fit()` adjusts weights using training data
- Controls the number of samples per gradient update
- Evaluates performance on separate data after each epoch

- Final evaluation demonstrates the model's ability to generalise well to unseen test data, confirming effective learning and minimal overfitting.

```

# 5. Evaluation\n",
"\n",
"Print out the test score (loss value) and accuracy of the model
on the test data."
},
"metadata": {
  "id": "qV1XWQcW8nKc"
},
{
  "cell_type": "code",
  "source": [
    "# Evaluate the model on the test data\n",
    "test_loss, test_acc = model.evaluate(x_test, y_test,
    verbose=2)\n",
    "print(f'Test accuracy: {test_acc}')\n"
  ],
  "metadata": {
    "colab": {
      "base_uri": "https://localhost:8080/"
    },
    "id": "yH8FVVCVbK4r",
    "outputId": "c86a2200-7831-4e63-d5e8-0f83f97cf30e"
  },
  "execution_count": 7,
  "outputs": [
    {
      "output_type": "stream",
      "name": "stdout",
      "text": [
        "313/313 - 4s - loss: 0.9150 - accuracy: 0.6737 - 4s/epoch -
        12ms/step\n",
        "Test accuracy: 0.6736999750137329\n"
      ]
    }
  ]
}
],

```

- Evaluate model on test set to assess performance
- Visualise training process to track accuracy and loss
- Identify signs of overfitting or underfitting
- "Model Accuracy" plot shows steady improvement in learning
- "Model Loss" plot confirms decreasing loss over epochs
- Indicates effective learning and generalisation to validation data

Applications of Deep Learning for NLP

- Deep learning enhances search engines by predicting user input and improving result accuracy through contextual understanding of queries.
- Real-time machine translation enables seamless cross-language communication, including instant translation of text from images using mobile devices.
- Sentiment analysis on social media supports targeted advertising by interpreting user emotions across posts in multiple languages.
- Applications such as chatbots, resume screening tools, and document summarisation streamline professional workflows and improve service efficiency.

Search predictions: Google predicts text for better accuracy and user experience

Machine translation: Tools like Google Translate provide real-time translation, enabling seamless global communication

Sentiment analysis: Analyses sentiment in posts to tailor targeted advertising based on user interactions

Chatbots: AI bots provide human-like assistance across industries

- Voice assistants, grammar tools, and email applications integrate deep learning to automate responses, correct language, and filter spam effectively.

Document classification: NLP models aid in quickly filtering and summarising documents, such as resumes, for hiring managers

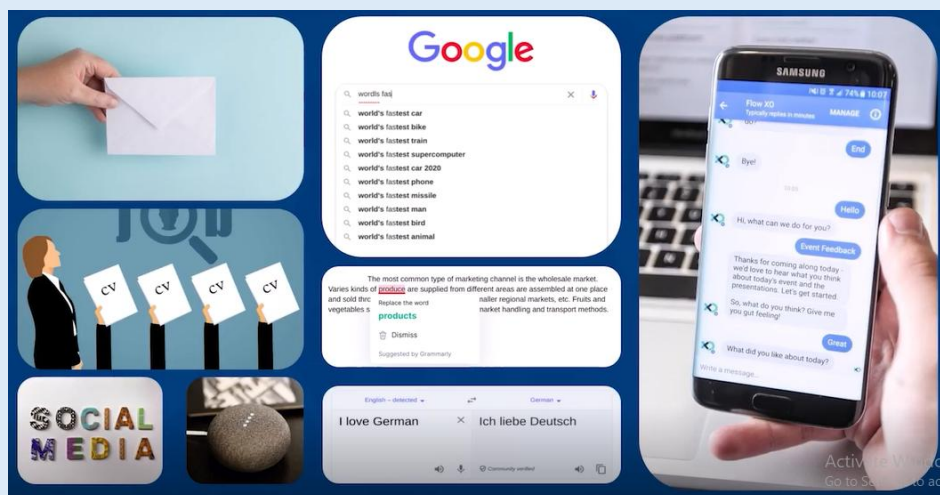
Voice assistants: Siri, Google Assistant and Alexa process speech for real-time responses

Grammar enhancement: Grammarly improves writing by correcting grammar and style

Email optimisation: Gmail filters spam and suggests responses for efficiency

Deep Learning for NLP, what is RNN and How it Works?

- Natural Language Processing enables machines to interpret and generate human language, powering applications like search engines, voice assistants, and real-time captioning.



- Recurrent Neural Networks (RNNs) are designed to process sequential data by maintaining short-term memory through feedback loops between time steps.

Sequential data processing: RNNs are designed for processing sequences, such as language or time series data, by maintaining memory over time.

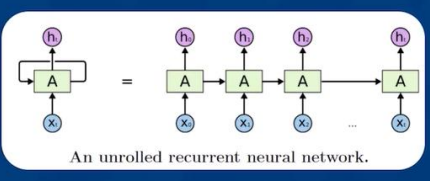
Predictive ability: RNNs predict the next element in a sequence based on previous inputs, using feedback from each step as input for the next.

Memory effect: RNNs recycle outputs back into the network, enabling them to remember previous data and make informed predictions.

Handling variable-length sequences: RNNs process variable-length input sequences, crucial for tasks like language translation or speech recognition where context matters.

An unrolled recurrent neural network.

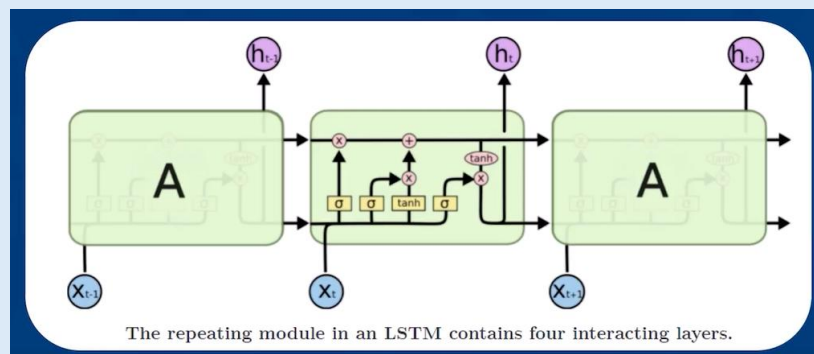
- RNNs face limitations with long-term dependencies due to the vanishing gradient problem, making them less effective for tasks requiring distant contextual understanding.



An unrolled recurrent neural network.

- Sequential data processing: RNNs process data step-by-step, passing outputs from one time step to the next to retain context
- Short-term dependencies: RNNs excel at remembering and utilising short-term dependencies in sequences, as seen in predicting the next word in a sentence
- Long-term dependency limitation: RNNs struggle with long-term dependencies, failing to retain information from earlier in long sequences
- Vanishing gradient problem: The vanishing gradient problem limits RNNs' ability to maintain long-term dependencies, reducing their effectiveness over long sequences

- Long Short-Term Memory (LSTM) networks improve upon RNNs by incorporating memory cells and gates that manage information flow across longer sequences.



- Text must be converted into numerical form for deep learning using techniques like one-hot encoding or word embeddings that capture semantic relationships in compact vectors.

Keras Example for Text Classification

- The IMDB dataset is used for sentiment analysis, containing movie reviews labelled as either positive or negative.

```

----- . . .
"# 1. Imports and Setup\n",
"\n",
"- Imports: Import TensorFlow and necessary modules from Keras\n",
"- Parameters: Set parameters such as max_features (maximum\n",
"number of words to consider as features in the vocabulary), maxlen\n",
"(maximum length of sequences), batch_size (number of samples per gradient\n",
"update), embedding_dims (dimensionality of the dense embedding), and\n",
"epochs (number of training iterations)."\n",
},
"metadata": {
  "id": "SxArTSWSAqBS"
},
{
  "cell_type": "code",
  "execution_count": null,
  "metadata": {
    "id": "exdx0sAhApCs"
  },
},

```

```

"outputs": [],
"source": [
  "# Importing necessary libraries\n",
  "import tensorflow as tf # Import TensorFlow library for deep\n",
  "learning operations\n",
  "from tensorflow.keras.datasets import imdb # Import IMDB\n",
  "dataset for sentiment analysis\n",
  "from tensorflow.keras.preprocessing.sequence import\n",
  "pad_sequences # Utility for padding sequences\n",
  "from tensorflow.keras.models import Sequential # Sequential\n",
  "model type for layer stacking\n",
  "from tensorflow.keras.layers import LSTM, Dense, Embedding #\n",
  "Different types of neural network layers\n",
  "from keras.utils import plot_model\n",
  "\n",
  "# Setting parameters for the model and training\n",
  "max_features = 20000 # Number of words to consider as features\n",
  "from the dataset\n",
  "maxlen = 200 # Cuts off texts after this number of words\n",
  "batch_size = 32 # Number of samples per gradient update during\n",
  "training\n",
  "embedding_dims = 128 # Dimensionality of the dense embedding\n",
  "space\n",
  "epochs = 5 # Number of training iterations over the entire\n",
  "dataset"\n",
],
{
  "cell_type": "markdown",
  "source": [

```

- Pre-processing includes restricting vocabulary size and padding sequences to ensure uniform input length for LSTM layers.

```

    "# 3. Preprocessing Data\n",
    "\n",
    "Use pad_sequences to ensure all sequences (x_train and x_test)\n",
    "have the same length (maxlen). This is necessary because LSTM networks\n",
    "require inputs to have the same length.\n",
    },
    "metadata": {
        "id": "wgHt6BbJfU"
    },
    },
    {
        "cell_type": "code",
        "source": [
            "# Pad sequences to ensure uniform length for input to LSTM\n",
            "print('Pad sequences (samples x time)')\n",
            "x_train = pad_sequences(x_train, maxlen=maxlen) # Pad or\n",
            "truncate sequences to a fixed length of maxlen\n",
            "x_test = pad_sequences(x_test, maxlen=maxlen) # Pad or\n",
            "truncate sequences in the test set to the same length\n",
            "print('x_train shape:', x_train.shape) # Output the shape of\n",
            "the padded training data\n",
            "print('x_test shape:', x_test.shape) # Output the shape of\n",
            "the padded test data\n",
        ],
        "metadata": {
            "colab": {
                "base_uri": "https://localhost:8080/"
            },
            "id": "M5-sL3reBlyI",
            "outputId": "8b1bdebe-138d-4655-9fc8-855f9555e0a0"
        },
        "execution_count": null,
        "outputs": [
            {
                "output_type": "stream",
                "name": "stdout",
                "text": [
                    "Pad sequences (samples x time)\n",
                    "x_train shape: (2000, 200)\n",
                    "x_test shape: (2000, 200)\n"
                ]
            }
        ]
    }
]

```

- The model architecture features an embedding layer followed by an LSTM layer and a sigmoid-activated output for binary classification.
- Parameters like embedding dimension, batch size, and number of epochs are defined to optimise training and model performance.
- Evaluation on test data reveals model accuracy and loss, offering insights into generalisation capability on unseen reviews.

Module Summary

- Deep learning surpasses traditional machine learning by automatically extracting features from large-scale unstructured data without manual engineering.
- Advances in hardware, data availability and training algorithms have fueled the rise of deep neural networks capable of complex representation learning.

Deep learning automates feature extraction and handling unstructured data efficiently.

Advances in computational power have fueled the rise of deep learning.

The availability of large datasets enables better model training and performance.

Improved training algorithms allow deep neural networks to learn complex representations.

- Convolutional neural networks (CNNs) are central to computer vision tasks, using layered architecture for image classification and feature extraction.
- Recurrent neural networks (RNNs) and long short-term memory (LSTM) models effectively handle sequential data for tasks like sentiment analysis and text classification.

- Keras tutorials demonstrate practical implementation of CNNs with CIFAR-10 and LSTMs with IMDB dataset, showcasing training workflows and model capabilities.

THE END